#.NET 10におけるNative AOTの技術的成熟度とパフォーマンス評価：提供資料の妥当性検証とアーキテクチャへの包括的示唆

1. 序論と提供資料の全体的評価

2025年11月に長期サポート（LTS）バージョンとしてリリースされ、2028年11月までサポートが提供される.NET 10は、クラウドネイティブアーキテクチャ、マイクロサービス、およびサーバーレスコンピューティングにおける実行効率を極限まで高めることを主眼に設計されたプラットフォームである[1]。このリリースにおいて、Ahead-of-Time（AOT）コンパイル技術は単なる実験的フェーズを脱却し、リソース制約の厳しい環境における第一級の選択肢として成熟した[1]。

本レポートは、ユーザーより提示された事前資料「.NET 10 Native AOT の技術的成熟度と実際のパフォーマンス改善」の記載内容を、Microsoftの公式ドキュメント、コンパイラの実装メカニズム、ベンチマーク結果、および開発者コミュニティにおける広範な実証データと照合し、その妥当性を多角的に検証するものである。包括的な分析の結果、提供資料が主張する「Native AOTによる起動時間の短縮とメモリ消費の大幅な削減」というパフォーマンスに関する定量的な見解は、実運用環境におけるデータと極めて高い精度で一致していることが確認された[3]。

しかしながら、提供資料の最も重大な主張である「Entity Framework Core（EF Core）およびMicrosoft.Data.SqlClientのAOT対応がほぼ完璧になった」という記述については、現在の技術的実態および公式ガイドラインと決定的な乖離が存在する[5]。本稿では、.NET 10の基盤となるJIT（Just-In-Time）コンパイラとNative AOTのアーキテクチャ上の相違を紐解きながら、エコシステム全体の互換性の実態、プロファイル主導最適化（PGO）によるパラダイムシフト、およびシステム移行に向けた戦略的ロードマップを詳細に論じる。

2. 実行アーキテクチャの進化とコンパイル基盤の刷新

.NETの伝統的な実行モデルは、中間言語（IL）を実行環境下で動的にネイティブコードへ変換するJITコンパイラに依存している[7]。これに対し、Native AOTモデルは、ビルド時にアプリケーションのILを特定のターゲットプラットフォーム（Linux x64、Windows x64、macOS Arm64など）向けのネイティブマシンコードに完全にコンパイルし、ランタイムを含まない自己完結型の小規模なバイナリを生成する[5]。.NET 10におけるNative AOTの技術的成熟度は、主に「事前初期化の強化」と「静的解析（トリミング）アルゴリズムの洗練」によって支えられている。

事前初期化（Preinitialization）のメカニズムと拡張

Native AOTがJITモデルに対して圧倒的な起動速度を誇るアーキテクチャ上の最大の理由は、ビルド時におけるコードの解釈と事前初期化のメカニズムにある。JIT環境では、アプリケーションの起動時やクラスへの初回アクセス時に、静的コンストラクタの実行やstatic readonlyフィールドの初期化処理がオーバーヘッドとして発生する[9]。Native AOTはこれらの初期化処理をビルドプロセス内で事前に評価し、その結果のメモリ状態を生成されるアセンブリ内にデータとして永続化する機能を持つ[9]。

.NET 10の事前初期化メカニズムは大幅に拡張され、これまでコンパイルの妨げとなっていた動的な評価の多くがビルド時に解決可能となった。具体的には、配列から生成されるSpan<T>のサポートが

追加され、.AsSpan()のようなメソッドの使用がビルドプロセスを中断させることがなくなった[9]。さらに、他の型の静的フィールドへのアクセス、ポインタの参照解決、独自に静的コンストラクタを持つ他の型に対するメソッド呼び出しに関する制限が撤廃されたことで、ランタイムにおいて実行される初期化コードが極限まで削減され、起動レイテンシが物理的な限界に近づいている[9]。

メタデータ分析とトリミングの厳格化

Native AOTモデルは、リフレクションや動的コード生成（System.Reflection.Emit）といった動的な型解決メカニズムと本質的に対立する[5]。JIT環境では未知の型を実行時にロード（Assembly.LoadFileなど）してインスタンス化することが可能であるが、Native AOT環境では、コンパイラがビルド時にアプリケーションが通過する可能性のあるすべてのコードパスと依存関係を静的に特定し、不要なコードを削除（トリミング）できなければならない[5]。

.NET 10では、このトリミングアルゴリズムが根本から強化された。重複するコードセグメントを統合するコードフォールディング（Code Folding）機能の改善や、インターフェース型のデータ保存手法の見直しにより、実行バイナリのサイズは.NET 9と比較して最大40%削減されている[4]。また、リフレクションを多用するアプリケーション向けに、どのメタデータをバイナリに含めるべきかを判断するアナライザーの精度が向上し、ディスク占有量が最適化されている[9]。

開発体験（DX）の観点において特筆すべきは、アセンブリメタデータとしてのIsAotCompatible属性の標準化である[5]。.NET 10をターゲット（net10.0）とするプロジェクトにおいて、開発者がプロジェクトファイルでVerifyReferenceAotCompatibilityプロパティを有効にすると、依存するNuGetパッケージや外部ライブラリ群がNative AOT環境で安全に動作するかどうかをコンパイラが厳密に検証する[5]。非互換性が検出された場合はコンパイル時に警告（IL3058）が発せられるため、提供資料が指摘する通り、実行時になって初めてリフレクション起因のエラーでクラッシュするという事態を未然に防ぐ開発ワークフローが確立されている[4]。

3. パフォーマンス指標の定量的検証とクラウドエコノミクス

提供資料に記載されたマイクロサービス（Web API）におけるJITとNative AOTのパフォーマンス比較データは、クラウドネイティブ環境におけるAOTの絶対的な優位性を示している。以下の表は、提供資料の数値を基に、公式ベンチマークおよび広範な実証データに基づくコンテキストを統合したものである。

評価指標	**JIT**コンパイルモデル（標準）	**Native AOT**モデル（**.NET 10**）	アーキテクチャ上の影響と経済的価値
起動時間 **(**コールドスタート**)**	200ms - 800ms	15ms - 50ms	80%〜90%以上の削減。トラフィックバーストに対する即時スケーリングを可能にし、ユーザーの

			待機時間を排除する[3]。実測値では170msから69msへの改善事例も報告されている[11]。
メモリ使用量 **(RSS)**	80MB - 120MB	25MB - 40MB	60%〜70%の削減。ガーベジコレクション（GC）の負荷低下およびKubernetes等におけるコンテナ集約度の飛躍的向上[3]。
実行バイナリサイズ	100MB以上（ランタイム依存）	4MB - 20MB	最大40%のサイズ削減。単一バイナリ化により、コンテナイメージの軽量化、ネットワーク転送量の削減、デプロイ時間の短縮を実現[4]。System.Text.Jsonソースジェネレーターの利用等により4MBまで縮小した報告も存在する[12]。
スループット **(RPS)**	基準値 (100%)	95% - 105%	静的PGO（MIBCプロファイル）の適用とコンパイル時の最適化により、JITのピークスループットと同等またはそれを凌駕する水準を達成[3]。

AWS LambdaおよびAzure Functionsにおけるコスト削減効果

コールドスタート時間の劇的な短縮とメモリ使用量の削減は、AWS LambdaやAzure Functionsといったサーバーレス・コンピューティング環境において、直接的な経済的価値を生み出す[4]。サーバーレス環境の課金モデルは、関数の実行時間（ミリ秒単位）と割り当てられたメモリ量に基づいて計算されるため、リソース消費の最適化はインフラストラクチャコストの削減に直結する。

実運用データに基づく報告によれば、EC2などの仮想マシンからAWS Lambdaベースのイベント駆動

型サーバーレスアーキテクチャへの移行に伴い、インフラストラクチャの運用コストが平均して70%削減され、ワークロードの変動が激しい環境においては60%から80%のコスト削減が達成されている[14]。.NET 10のNative AOTを適用し、AWS Graviton3（ARM64アーキテクチャ）上で実行した場合、コールドスタート時間は50ms未満に抑えられ、プロビジョニングされた同時実行（予約済みインスタンス）を維持する必要性がほぼ完全に排除される[3]。提供資料の「コストが40%削減できた」という現場の評価は、アーキテクチャのスケールゼロ（トラフィック非発生時のリソース完全解放）を可能にするNative AOTの特性によって理論的かつ実証的に裏付けられている[3]。

ARM32/ARM64アーキテクチャにおけるエッジコンピューティングの革新

クラウド環境のみならず、リソースの制約が厳しいエッジデバイスやIoT機器においてもNative AOTは極めて重要な役割を果たす。.NET 9で導入された32ビットARMアーキテクチャ向けのNative AOTサポートは、.NET 10においてさらに成熟し、0.8GHz駆動のシングルコアARMプロセッサを搭載した旧式の組み込みデバイスにおいても、フル機能の.NETアプリケーションが瞬時に起動し、ネイティブC++アプリケーションと同等のパフォーマンスを発揮する事例が報告されている[12]。JITコンパイラのウォームアップにかかる計算資源の消費を排除できるため、CPUクロックやメモリ帯域幅に制限のあるハードウェア環境において、Native AOTは運用可能性を根本から変革する技術となっている。

4. ランタイム最適化とJITコンパイラの進化がAOTに与える影響

Native AOTは実行時にJITコンパイラを持たないが、ビルドプロセスにおいてはJITコンパイラのコード生成エンジンを活用してネイティブマシンコードを生成する。したがって、.NET 10におけるJITコンパイラの最適化アルゴリズムの進化は、ほぼそのままNative AOTの実行速度の向上へと波及する[9]。.NET 10では、高負荷・高スループット環境のパフォーマンスを底上げする複数の高度な最適化が導入された。

エスケープ解析の拡張とスタックアロケーション

.NET 10における最も破壊的なパフォーマンス改善の一つは、エスケープ解析（Escape Analysis）の適用範囲の拡大に基づくスタックアロケーションの強化である[16]。エスケープ解析とは、コンパイラがメソッド内で生成されたオブジェクトの生存期間を静的に追跡し、そのオブジェクトがメソッドのスコープを越えて外部に参照されない（エスケープしない）ことを証明する技術である。

JITコンパイラがオブジェクトがエスケープしないと証明できた場合、そのオブジェクトは通常のマネージドヒープではなく、実行スレッドのコールスタック上に直接割り当てられる[16]。スタックアロケーションは、単なるポインタの移動のみでメモリを確保でき、メソッドの終了と同時に自動的にメモリが解放されるため、ヒープアロケーションと比較して圧倒的に計算コストが低い[16]。さらに重要な点として、スタックに割り当てられたオブジェクトはガーベジコレクタ（GC）による追跡対象から外れるため、高トラフィックなAPI環境においてアプリケーションのレイテンシスパイクの原因となるGCの一時停止時間（Pause Time）を根本的に削減する[16]。.NET 9で導入された初期のエスケープ解析と比較して、.NET 10ではより複雑な制御フローにおける追跡が可能となり、業務アプリケーションコードの広範な領域でヒープアロケーションの削減が実現している[16]。

構造体の物理的プロモーションとループ最適化

.NET 10のJITコンパイラは、構造体（Struct）の引数渡しに関するコード生成を大幅に改善した。「物理的プロモーション（Physical Promotion）」と呼ばれる最適化手法により、コンパイラは構造体を単一のメモリブロックとして扱うのではなく、構造体に含まれる各メンバー変数を個別のCPUレジスタに直接割り当てることを試みる[17]。これにより、メモリへのストアおよびロード命令が排除され、メソッド呼び出し時のオーバーヘッドが劇的に減少する。

また、ループの反転（Loop Inversion）や境界条件チェック（Bounds Checking）の排除アルゴリズムも高度化されており、配列やSpan<T>に対する反復処理がベクトル化命令（.NET 10でサポートが強化されたAVX10.2など）へとシームレスに変換されやすくなっている[9]。

LINQのパフォーマンス向上とサイズ最適化のトレードオフ

データの反復処理とクエリ構築において不可欠なLINQ（Language Integrated Query）も、.NET 10において大幅な最適化の恩恵を受けている。以下のベンチマーク表は、Skip().Take().ToList()という一般的なページング処理のパターンにおける、.NET 9と.NET 10の実行速度の比較を示している[9]。

メソッド / 処理内容	実行環境	平均実行時間 **(Mean)**	改善比率 **(Ratio)**
SkipTakeToList	.NET 9.0	1,218.9 ns	1.00
SkipTakeToList	.NET 10.0	257.4 ns	0.21 (約80%の高速化)
Sum (配列の集計)	.NET 9.0	19.530 ns	1.00
Sum (配列の集計)	.NET 10.0	6.685 ns	0.34 (約65%の高速化)

これらの劇的な高速化は、内部実装において状態マシンのアロケーションが最適化され、可能な限りSpan<T>ベースのベクトル化されたコピー命令（CopyTo）へと処理が委譲されるようになった結果である[9]。

ただし、Native AOT環境においては特有のトレードオフが存在する。長年にわたるLINQのパフォーマンス改善は、基底となるIterator<T>クラスから多数の特殊化された反復子クラスを派生させることで実現されてきた[9]。JIT環境ではこれらの特殊化されたクラスがフルに活用されるが、Native AOT環境では、例えばEnumerable.Containsを一度でも呼び出すと、トリミングアルゴリズムがそれらの特

殊化された実装を全てバイナリに含めざるを得なくなり、アセンブリサイズが非自明に肥大化するという問題が生じる[9]。

この問題を回避するため、Native AOTビルド（<PublishAot>true</PublishAot>を指定した場合）では、デフォルトでUseSizeOptimizedLinqオプションが有効化され、過去10年間の高度な速度最適化コードを意図的に排除した「サイズ最適化版」のSystem.Linq.dllがリンクされる[9]。この挙動によりバイナリサイズは最小化されるが、特定のデータ処理ループにおいて.NET 10のNative AOTビルドがJITビルドと比較して顕著なパフォーマンス後退（Regression）を示すケースがコミュニティから報告されている[19]。これは、AOTの導入にあたってサイズとスループットのトレードオフを開発者が明示的に評価する必要があることを示唆している。

5. プロファイル主導最適化（PGO）のパラダイムシフトと運用の実態

提供資料において、Native AOTのパフォーマンスをJITと同等水準まで引き上げる「鍵」として言及されているのが、プロファイル主導最適化（PGO: Profile-Guided Optimization）である[3]。AOTコンパイルの歴史的かつ構造的な弱点は、コンパイラが「コードが実行時にどのように呼び出されるか」というコンテキストを持ち得ないことにあった。PGOはこの限界を打破するためのメカニズムであるが、.NET 10においては「動的PGO」と「静的PGO」という二つの異なるアプローチがアーキテクチャに応じて提供されている。

階層型コンパイルと動的PGO（Dynamic PGO）の成熟

JITコンパイラ環境（CoreCLR）において標準で有効化されているのが、動的PGOである[20]。動的PGOは、開発者の事前の設定やビルド時の複雑なプロセスを一切必要とせず、ランタイムが自律的に実行時のアプリケーションの振る舞いを学習し、コードを最適化する[21]。

このメカニズムは、階層型コンパイル（Tiered Compilation）に依存している。アプリケーションの起動直後、JITは最適化をほとんど行わずに高速にネイティブコードを生成する（Tier 0コンパイル）[22]。このTier 0コードには、メソッドの呼び出し回数、ループの反転回数、インターフェース呼び出しの際に実際に利用された具象型などを記録するための計装（Instrumentation）プローブが埋め込まれている[21]。

アプリケーションが実行を続け、特定のメソッドが頻繁に呼び出される「ホットパス」であることが判明すると、JITコンパイラはバックグラウンドスレッドでそのメソッドを再コンパイル（Tier 1）する[22]。この際、JITは収集したプロファイルデータを利用して、極めて攻撃的な最適化を実行する。代表的なものが「保護された脱仮想化（Guarded Devirtualization: GDV）」である[9]。インターフェースを介したメソッド呼び出しは通常、仮想関数テーブルを検索するオーバーヘッドを伴うが、プロファイルデータから特定の具象型が99%の確率で呼び出されることが判明している場合、JITは型チェック（Guard）を挿入した上で、対象のメソッドを直接インライン化する[9]。

さらに、.NET 10では、Tier 0コードの実行中に長時間稼働するループ処理を動的にTier 1の最適化コードへ差し替える「オンスタックリプレイスメント（On-Stack Replacement: OSR）」のメカニズムが改善された。従来、OSRメソッドにはPGOの計装データが十分に付与されていなかったが、.NET 10で

はプロファイル合成アルゴリズムが強化され、IEnumerator<T>.Disposeのようなメソッドの末尾呼び出しに対しても正確な脱仮想化とインライン化が適用可能となった[9]。

静的PGO（Static PGO）のメカニズムとNative AOTへの適用

Native AOTアプリケーションは実行時にJITコンパイラを持たないため、上記の動的PGOを利用することは物理的に不可能である[5]。AOT環境においてJITと同等のスループットを達成するためには、ビルド時に実行プロファイルをコンパイラへ提供する「静的PGO」の導入が不可欠となる[13]。

静的PGOを利用するワークフローは以下のステップで構成される。

1. アプリケーションにプロファイル収集用の計装コードを埋め込んだ特別なビルドを作成する[21]。
2. そのビルドに対して、本番環境のトラフィックを正確に模倣した代表的なシナリオテストを実行する[21]。
3. テスト実行中に収集された実行データを、MIBC（Managed Intermediate Binary Code）形式のプロファイルファイルとして抽出する[13]。
4. 最終的なNative AOTのリリースビルドを実行する際、コンパイラ（ILC）にこのMIBCファイルを引数として渡し、コードレイアウトやインライン化の意思決定に反映させる[27]。

提供資料が指摘する通り、静的PGOが正しく適用された場合、AOTコンパイラはJITのようなコンパイル時間の制約（ミリ秒単位での処理完了要求）を受けないため、プログラム全体の最適化（Whole Program Optimization）というより高次元の解析を実行できる[26]。これにより、JITのピーク性能を上回る実行効率を達成するケースも実在する。

静的PGO運用の実態とコミュニティが直面する課題

理論上は強力な静的PGOであるが、技術コミュニティからのフィードバックを詳細に分析すると、その運用には未だ高い障壁が存在することが明らかになる。

多くの開発者が、Native AOTビルドにおいてMIBCファイルをコンパイラに引き渡したにもかかわらず、そのプロファイルデータが完全に無視され、生成されたバイナリが静的PGO未適用のものと同一のパフォーマンス（JITと比較して20%から25%程度のパフォーマンス後退）しか発揮しないという問題に直面している[27]。この問題の原因は、crossgen2やコンパイラドライバに対する複雑なパラメータ渡し（--embed-pgo-dataや--mibc引数の指定）の設定不備、あるいは収集されたプロファイルデータの質がコンパイラの要求する水準を満たしていないことに起因するケースが多い[27]。

動的PGOが「コードを記述して実行するだけで自動的に最高性能を引き出す」ゼロエフォートのアプローチであるのに対し、静的PGOはCI/CDパイプラインにおける高度なビルド自動化インフラストラクチャと、本番環境を正確に反映したテストシナリオの維持を開発チームに要求する[21]。したがって、静的PGOは優れたパフォーマンスを提供するものの、その実装難易度を考慮すると、提供資料が示唆するほど気軽に導入できるソリューションとは言い難いのが実情である。

6. 提供資料の致命的誤謬：Entity Framework Core 10のAOT互換性と制約

本レポートにおいて最も強く訂正を促すべき点は、提供資料における「Entity Framework Core（EF Core）のAOT対応がほぼ完璧になり、実行時エラーに怯える必要がなくなった」という主張である[3]。公式ドキュメント、リリースノート、および開発チームによる議論を総合的に分析した結果、この主張は技術的事実と完全に矛盾しており、極めて危険な過大評価であると結論づけられる。

EF Core 10におけるNative AOTの公式ステータス

Microsoftの公式ドキュメントは、EF CoreにおけるNative AOTサポートおよび事前コンパイル機能について、現時点では「極めて実験的（highly experimental）」な性質のものであり、「EF NativeAOTアプリケーションを本番環境（Production）にデプロイすることは推奨しない（recommend against）」と明確な警告を発している[5]。現在の実装は、将来のメジャーリリース（コミュニティの推測によれば.NET 12のタイムラインである2027年末）に向けたインフラストラクチャの試験的導入という位置づけに過ぎない[5]。

動的コード生成の壁と事前コンパイルの限界

EF CoreのAOT対応がこれほどまでに困難である理由は、同フレームワークの根本的な設計思想に由来する。EF Coreは、アプリケーションコード内に記述されたLINQクエリを実行時に式ツリー（Expression Tree）として動的に解析し、対象のデータベース（SQL Server、PostgreSQLなど）の特性に合わせたSQLクエリをオンザフライで生成・コンパイルする機能に依存している[5]。Native AOT環境は、実行時におけるIL（中間言語）コードの生成や動的コンパイルを一切許可しないため、EF Coreの心臓部とも言えるこの動的クエリ変換メカニズムはAOTと完全に非互換である[10]。

このアーキテクチャ上の制約を回避するため、EF Coreチームは「クエリの事前コンパイル（Precompiled Queries）」というアプローチを導入した[5]。開発者はdotnet ef dbcontext optimize --precompile-queries --nativeaotというコマンドを実行することで、ソースコード内に記述されたLINQクエリをビルドプロセス内で静的解析し、C#インターセプターという実験的な言語機能を活用して、実行時のクエリ生成を回避する静的コードを自動生成する[5]。

しかし、この事前コンパイルインフラストラクチャには、実世界のエンタープライズアプリケーション開発において致命的となる以下の制約が存在する。

制限事項	影響範囲とアーキテクチャ上の課題
動的クエリ（**Dynamic LINQ**）の非互換性	ユーザーの入力や実行時の条件分岐に基づいて.Where句や.OrderBy句を動的に結合するクエリは、ビルド時に静的解析できないためコンパイルエラーとなる[5]。検索フィルターや複雑なダッシュボードクエリの大半がこの制約に抵触する。
LINQクエリ構文の非サポート	メソッドチェーン（.Select().Where()など）ではなく、クエリ包含構文（from x in y select z）で

	記述されたLINQは現在未対応である[5]。
値コンバーターと状態のキャプチャ	ドメイン駆動設計（DDD）などで多用される、キャプチャされた状態を持つカスタム値コンバーターの静的コード生成がサポートされていない[5]。
コードの肥大化と生成時間の増大	すべてのクエリバリエーションに対して静的なインターセプタークラスを生成するため、コンパイル時間が非現実的な長さに達し、生成されるコードのサイズも膨大になる[5]。

コミュニティからも、「公式ドキュメントに記載された極めてシンプルなクエリでさえ、事前コンパイルコマンドを実行するとDynamic LINQ queries are not supportedという例外が発生して失敗する」という苦情が多数寄せられており、Native AOT環境下でのEF Coreの運用は、現時点では実用に耐えうるレベルに到達していない[6]。したがって、提供資料が主張する「ほぼ完璧になった」という表現は誤りであり、「極めて限定的な静的クエリにのみ対応した実験段階である」と認識を改める必要がある。

EF Core 10の真の強み：複雑な型とJSON/Vector統合

AOT対応においては苦戦を強いられているEF Coreであるが、JIT環境下におけるORMとしての進化は目覚ましいものがある。提供資料の誤りを指摘する一方で、EF Core 10がもたらした真の技術的革新を整理しておくことは、適切なアーキテクチャ選定において不可欠である。

1. **SQL Server 2025**向けのネイティブ**JSON**サポート: EF Core 10は、SQL Server 2025およびAzure SQL Databaseで導入されたネイティブなjsonデータ型を完全にサポートしている。複雑な型（Complex Types）やプリミティブなコレクション（intなど）をJSONカラムに直接マッピングし、データベース側でインデックスを効かせた効率的なJSON検索を実行することが可能となった[29]。既存のnvarcharベースのJSONカラムは、移行スクリプトを通じて自動的に新しいjson型へアップグレードされる機能も備えている[29]。
2. **Vector**検索と**AI**ワークロードの統合: Azure Cosmos DB for NoSQLおよびSQL Server向けのベクトル近似検索機能がプレビューを脱して本番利用可能（Production Ready）となった。SqlVector<float>型のプロパティをエンティティに定義し、EF.Functions.VectorDistanceを利用することで、生成AIにおけるRAG（Retrieval-Augmented Generation）アーキテクチャのセマンティック検索を、外部のベクトルデータベースに依存することなく同一のトランザクション境界内で実行できるようになった[2]。Cosmos DBにおいては、フルテキスト検索とベクトル検索を組み合わせたハイブリッド検索（Reciprocal Rank Fusion）もサポートされている[29]。
3. クエリ表現と一括更新の高度化: 長らく要望の強かったLeftJoinおよびRightJoin演算子がネイティブに導入され、複雑な外部結合を直感的なLINQで記述可能となった[2]。また、論理削除（Soft Delete）やマルチテナントアーキテクチャで必須となるグローバルクエリフィルターについて、IgnoreQueryFiltersに特定のフィルター名を指定することで、個別にオン・オフを制御できる「名前付きクエリフィルター」機能が実装された[18]。さらに、何万件ものレコードをメモリに読み込むことなくデータベース上で直接更新・削除するExecuteUpdateAsyncおよび

ExecuteDeleteAsyncメソッドにおいて、従来の式ツリーだけでなく、通常のラムダ式による条件指定がサポートされ、開発者の生産性が大きく向上している[30]。

これらの強力な機能群は、実行時の動的解析（JITコンパイルモデル）を前提として高度に最適化されている。したがって、高度なデータモデリングと動的検索を要求するエンタープライズアプリケーションにおいては、Native AOTの採用を見送り、引き続きJITモデルの下でEF Coreの全機能を活用することが、戦略的に最も正しい選択となる。

7. データアクセス層の実態：Microsoft.Data.SqlClient 7.0と代替手法

EF Coreの背後でリレーショナルデータベースとの通信を担う基盤プロバイダであるMicrosoft.Data.SqlClientについても、Native AOTとの互換性に関しては慎重な評価が求められる。

SqlClient 7.0におけるAOT対応の課題

.NET 10のリリースサイクルに合わせて提供されているMicrosoft.Data.SqlClient 7.0では、内部実装におけるTDSパーサーのパフォーマンス向上や、オープンソースの可観測性標準であるOpenTelemetryへの対応など、クラウドネイティブ向けの近代化が進められている[33]。また、リリースノートには「Native AOTサポートの改善」が含まれており、コア部分のコンパイル警告の解消に向けた努力が確認できる[33]。

しかしながら、実運用環境でのAOT対応は依然として道半ばである。歴史的経緯から、ADO.NETプロバイダはデータセットとC#オブジェクトのマッピングにおいて、System.Reflection.Emitによる動的なIL生成に依存してきたというアーキテクチャ上の負債を抱えている[10]。また、ネイティブSNI（Server Network Interface）DLLのロードに関する問題や、Azure Active Directory（現 Entra ID）認証を利用する際の互換性問題がコミュニティで多数報告されている[35]。

特に、バージョン7.0からEntra ID認証に関連する機能がMicrosoft.Data.SqlClient.Extensions.Azureパッケージへと完全に分離され、ActiveDirectoryPasswordなどの認証メソッドが非推奨（Obsolete）となる破壊的変更が行われた[36]。この分離はライブラリの軽量化に貢献する一方で、拡張パッケージ側でのリフレクションベースの依存関係解決がNative AOT環境でクラッシュを引き起こすという回帰バグ（Regression）を生み出しており、高度な認証基盤を必要とするクラウドアプリケーションのAOT移行において大きなブロッカーとなっている[35]。

Pragmaticな解決策：Dapper.AOTの台頭

EF CoreおよびSqlClientがAOT環境で苦戦を強いられる中、データベースアクセスを伴うマイクロサービスやサーバーレス関数をNative AOTで構築するための最適解として、開発者コミュニティから圧倒的な支持を集めているのが「Dapper.AOT」である[6]。

従来のDapperは、純粋なADO.NETコードの冗長性を排除する軽量なマッパーとして広く普及していたが、その極めて高速なマッピング処理の裏側には、実行時の動的IL生成アルゴリズムが存在していた[10]。これはNative AOTの設計原則と真っ向から衝突するため、従来のDapperをAOT互換に改

修することは不可能であった[10]。

この課題を解決するために再設計されたのがDapper.AOTである。Dapper.AOTは、実行時ではなく「コンパイル時」にRoslynソースジェネレーターを活用してマッピングコードを静的に生成するアプローチを採用した[10]。これにより、リフレクションや動的コード生成を完全に排除しつつ、従来のDapperと同等の優れた開発体験（DX）と超高速な実行速度を維持している。現時点でデータベースと通信するAPIをNative AOTで本番運用する場合、EF Coreに見切りをつけ、Dapper.AOTを採用することが事実上の業界標準（De facto standard）となっている[6]。

8. Webフレームワークとフロントエンドにおける**AOT**の成熟度

データアクセス層の課題とは対照的に、Webフレームワーク（ASP.NET Core）およびフロントエンド（Blazor、MAUI）の領域においては、.NET 10によるNative AOTサポートは目覚ましい成熟度を示しており、提供資料の肯定的な評価を完全に裏付けている。

ASP.NET Core Minimal APIsとJSONソースジェネレーターの進化

ASP.NET Coreにおいて、従来のMVCモデル（コントローラーベースのルーティング）は、実行時の属性（Attribute）探索や広範なリフレクションに依存するため、AOT環境には適していない[38]。これに対する解答として導入された「Minimal APIs」は、.NET 10においてAOT制約下でのルーティングとリクエストパイプラインの最適化が極限まで洗練された[39]。

提供資料が「ソースジェネレーターの進化」として言及している点は、特にJSONシリアライゼーションにおいて顕著な効果を発揮している[3]。System.Text.Jsonライブラリは、従来のリフレクションベースのプロパティ走査を廃止し、JsonSerializerContextを介してコンパイル時にシリアライズ・デシリアライズの処理コードを静的に生成する仕組みへと移行した[4]。.NET 10では、重複するJSONプロパティの厳格な検証や、よりメモリ効率に優れたPipeReaderのネイティブサポートが追加され、高トラフィック環境でのメモリ割り当てをさらに削減している[18]。

また、Minimal APIにおける特筆すべき進化として、OpenAPI（Swagger）仕様のドキュメント自動生成がNative AOT環境で標準サポートされるようになった[37]。.NET 10のプロジェクトテンプレート（webapiaot）では、Microsoft.AspNetCore.OpenApiパッケージがデフォルトで組み込まれ、ソースジェネレーターを通じてコンパイル時にAPIエンドポイントのメタデータを静的に抽出・生成する[40]。これにより、開発者は「AOTの制約」を理由にAPIドキュメントの自動生成という利便性を犠牲にする必要がなくなった。

これらの最適化により、ASP.NET Core Minimal APIsとDapper.AOTを組み合わせたマイクロサービスは、コールドスタートが事実上ゼロに等しく、メモリ消費量が数十MBで推移する、超高効率なクラウドネイティブアーキテクチャの完成形を提供している[15]。

Blazor WebAssemblyと.NET MAUIにおける**AOT**適用

Webブラウザ上でC#コードを実行するBlazor WebAssemblyにおいて、Native AOTコンパイル（C#からWebAssemblyバイナリへの事前コンパイル）は、長らく実験的機能として位置づけられてきたが、.NET 10においてついに「本番利用可能（Production Ready）」という公式ステータスを獲得した[41]。インタラクティブWebAssemblyモードや自動レンダリングモードを使用するBlazorアプリケーション

に対し、AOTコンパイル後のILトリミングアルゴリズムが強化されたことで、WebAssemblyバンドルのサイズ肥大化という長年の課題が克服され、クライアント側での圧倒的な描画速度と実行パフォーマンスを実現している[18]。

一方、クロスプラットフォームのモバイルおよびデスクトップUIフレームワークである.NET MAUIにおいては、ターゲットプラットフォームごとに異なるコンパイル戦略が採用されている[13]。AndroidやiOSなどのモバイルプラットフォームでは、歴史的にフットプリントが小さくAOTとJITの双方に優れたMonoランタイムが標準利用されてきた[13]。しかし、.NET 10からはデスクトップ・サーバー向けの高機能なCoreCLRランタイムがAndroid向けの実験的オプションとして導入され、.NET 11ではリリースビルドにおける標準ランタイムとなる計画が進行している[13]。これにより、モバイル開発においても階層型コンパイルや高度なプロファイル主導最適化（PGO）の恩恵をフルに享受できる道が開かれつつある[13]。

9. モノリシックシステムからの移行障壁とReadyToRun（R2R）の活用

提供資料は、Native AOTの利用を「巨大なモノリス（ビルド時間が長すぎる）」に対して非推奨としているが[3]、これは極めて実践的かつ妥当な助言である。エンタープライズ領域におけるアプリケーション近代化のプロセスにおいて、既存の巨大なコードベースをAOT環境へ移行する際には、ビルド時間以外のアーキテクチャ上の深刻な障壁が存在する。

###.NET Frameworkからの移行におけるエッジケース 2025年末現在においても、多くの企業システムは.NET 9や.NET 8、あるいは旧来の.NET Framework（4.x）上で稼働している[1]。これらのレガシーアプリケーションを.NET 10へ移行するだけでも大きな労力を要するが、さらにNative AOT化を目指す場合、克服不可能な「削除された技術」の壁に直面する[42]。

例えば、状態保持型のイベント駆動モデルであるASP.NET Web Forms、ワークフローオーケストレーションを担うWindows Workflow Foundation（WF）、プロセス間通信のための.NET Remotingといった技術は、モダンな.NET（Core）アーキテクチャには引き継がれておらず、直接的な代替手段が存在しない[42]。これらの技術に依存する数百のプロジェクトから構成されるモノリシックシステムは、BlazorやASP.NET Core MVCへの全面的なアーキテクチャ再構築（Rewrite）を強いられる。

さらに、モノリスシステムが依存する数百のサードパーティ製NuGetパッケージのうち、たった一つでもリフレクションに強く依存し（IsAotCompatibleメタデータを持たない）、トリミング警告（IL2026）を発するパッケージが存在すれば、アプリケーション全体のAOTコンパイルは安全に実行できなくなる[5]。このようなシステムにおいては、Native AOTの追求は非現実的であり、移行プロジェクト自体を頓挫させるリスクが高い。

中間解としてのReadyToRun (R2R) コンパイル

Native AOTの制約とトリミングのリスクを回避しつつ、コールドスタート時間の短縮という恩恵を得るための現実的な中間解（Middle ground）として推奨されるのが、ReadyToRun（R2R）コンパイルである[6]。

R2Rは、アプリケーションのアセンブリ内に、ターゲットプラットフォーム向けのネイティブマシンコードと標準の中間言語（MSIL）の両方をパッケージングする事前コンパイル形式である[13]。実行時、JITコ

ンパイラは既にコンパイル済みのネイティブコードが存在する場合はそれを直接実行し、欠落している最適化パスやR2Rでコンパイルできなかった部分については、従来通りJITによって動的にコンパイルを行う[13]。

.NET MAUIなどの環境では、このR2R技術が「複合的な部分R2R（Composite Partial ReadyToRun）」としてデフォルトで活用されている[13]。これは、複数のアセンブリを統合して事前コンパイルすることでクロスアセンブリの最適化を効かせつつ、MIBCプロファイルデータに基づいて「アプリケーションの起動と主要な動作にクリティカルなメソッド」のみを選択的にR2Rコンパイルし、それ以外のメソッドはJITやインタープリタに委ねる手法である[13]。

モノリシックなWeb APIアプリケーションにおいても、JITモデルの柔軟性（EF Coreの完全サポート、動的ローディングの許容）を維持したまま、<PublishReadyToRun>true</PublishReadyToRun>を指定することで、起動時間の大幅な短縮を安全に実現できる[6]。これは、Native AOTへの全面移行が困難なシステムにとって、リスクとリターンのバランスが最も取れた戦略的選択肢となる[6]。

10. 結論と戦略的アーキテクチャのロードマップ

本レポートでの検証を通じて、提供資料「.NET 10 Native AOT の技術的成熟度と実際のパフォーマンス改善」の記述内容は、パフォーマンス改善の定量的な成果やインフラストラクチャに対する経済的価値の評価においては極めて正確であり、現在の技術トレンドを的確に捉えていることが確認された。起動時間の90%削減やメモリフットプリントの大幅な縮小は、クラウドプロバイダーのインフラ運用コストを直接的に引き下げる破壊的なイノベーションである[3]。

しかし、エンタープライズデータアクセス層の中核を成すEntity Framework CoreおよびMicrosoft.Data.SqlClientのAOT互換性に関する主張は致命的に誤っており、これを鵜呑みにしたアーキテクチャ設計はプロジェクトに重大な破綻をもたらすリスクがある[5]。

以上の技術的実態に基づき、.NET 10におけるアプリケーションアーキテクチャの選定に関する戦略的ロードマップを以下に提言する。

1. **Native AOT**を積極的に採用すべき領域
   - サーバーレス関数（**AWS Lambda / Azure Functions**）: コールドスタート問題の解消と課金コストの最小化において、Native AOTは圧倒的なROIを提供する[14]。
   - 高密度なマイクロサービス: Kubernetesクラスター等に展開される小規模なバックエンドAPI群。データアクセス層にはEF Coreではなく、静的コード生成に基づくDapper.AOTを採用し、ASP.NET Core Minimal APIsと組み合わせる構成を標準とする[37]。
   - **CLI**ツールおよびエッジコンピューティング:.NETランタイムを前提としない、完全な自己完結型バイナリが必要なリソース制約環境[12]。
2. **JIT**コンパイル（**CoreCLR**）を継続利用すべき領域
   - 複雑なドメインモデルを持つエンタープライズシステム: 動的LINQによる高度な検索フィルターや、EF Core 10で拡張されたネイティブJSON検索、ベクター検索機能に深く依存するアプリケーションは、JITモデルの動的解析能力が不可欠である[5]。
   - レガシー移行の途上にあるモノリス: 依存するライブラリ群のAOT互換性が完全に担保されるまでは、トリミングによる実行時クラッシュのリスクを避けるため、JITモデルを維持する。起動速度の改善が必要な場合は、中間解としてReadyToRun（R2R）コンパイルの導入

を優先する[6]。

.NET 10におけるNative AOTは、クラウドネイティブ開発における強力な武器として確立された。しかし、それは「銀の弾丸」ではなく、ソフトウェアの設計原則とトレードオフを厳格に要求する特化型のプラットフォームである。技術の限界を正確に把握し、対象となるワークロードの特性に合わせてAOTとJITを適材適所で使い分けることこそが、次世代の.NETシステムを成功に導くための唯一の道標となる。

引用文献

1. .NET 10 vs .NET 9 What Developers Should Choose in 2026 - Digisoft Solution, 5月 10, 2026にアクセス、https://www.digisoftsolution.com/blog/dotnet-9-vs-dotnet-10
2. Best 30 C# Project Ideas for Final Year Students (2026) - Codersera, 5月 10, 2026にアクセス、https://codersera.com/blog/best-30-c-project-ideas-for-final-year-students/
3. NET 10 Native AOT の技術的成熟度と実際のパフォーマンス改善を、開発者コミュニ.... pdf
4. .NET 10 Performance Improvements That Actually Matter in Production - WireFuture, 5月 10, 2026にアクセス、https://wirefuture.com/post/net-10-performance-improvements-that-actually-matter-in-production
5. NativeAOT Support and Precompiled Queries (Experimental) - EF ..., 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/ef/core/performance/nativeaot-and-precompiled-queries
6. Microsoft's own EF Core docs literally say "recommend against deploying EF NativeAOT applications in production" : r/dotnet - Reddit, 5月 10, 2026にアクセス、https://www.reddit.com/r/dotnet/comments/1sg48zk/microsofts_own_ef_core_docs_literally_say/
7. .NET 10 Runtime Performance: JIT, GC & NativeAOT Changes (2026), 5月 10, 2026にアクセス、https://codingdroplets.com/what-s-new-in-net-10-runtime-performance-jit-gc-and-nativeaot-changes-enterprise-teams-should-know
8. Native AOT deployment overview - .NET | Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/
9. Performance Improvements in .NET 10 - Microsoft Developer Blogs, 5月 10, 2026にアクセス、https://devblogs.microsoft.com/dotnet/performance-improvements-in-net-10/
10. How to make libraries compatible with native AOT - .NET Blog, 5月 10, 2026にアクセス、https://devblogs.microsoft.com/dotnet/creating-aot-compatible-libraries/
11. Native AOT in .NET 10: Everything for C# Developers - DEV Community, 5月 10, 2026にアクセス、https://dev.to/chandana_pushpakumara_4bf/native-aot-in-net-10-everything-for-c-developers-2m7e

12. blown away by .NET10 NativeAOT : r/dotnet - Reddit, 5月 10, 2026にアクセス、https://www.reddit.com/r/dotnet/comments/1r9uguu/blown_away_by_net10_nativeaot/
13. Runtimes and compilation in .NET MAUI - Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/dotnet/maui/deployment/runtimes-compilation?view=net-maui-10.0
14. AWS Serverless Computing Services - Futran Solutions, 5月 10, 2026にアクセス、https://futransolutions.com/services/aws-cloud/serverless-computing/
15. Surviving Native AOT: The Reflection Migration Guide Every .NET Architect Needs, 5月 10, 2026にアクセス、https://blog.stackademic.com/surviving-native-aot-the-reflection-migration-guide-every-net-architect-needs-fa3760fbb41b
16. Performance Improvements in .NET 10 : r/dotnet - Reddit, 5月 10, 2026にアクセス、https://www.reddit.com/r/dotnet/comments/1ndemz5/performance_improvements_in_net_10/
17. What's new in .NET 10 runtime - Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/runtime
18. What's new in .NET 10 - Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
19. Massive performance regression .NET 9 vs .NET 10 · Issue #115033 · dotnet/runtime, 5月 10, 2026にアクセス、https://github.com/dotnet/runtime/issues/115033
20. dotnet-claude-kit/knowledge/dotnet-whats-new.md at main - GitHub, 5月 10, 2026にアクセス、https://github.com/codewithmukesh/dotnet-claude-kit/blob/main/knowledge/dotnet-whats-new.md
21. Performance Improvements in .NET 8 - Microsoft Developer Blogs, 5月 10, 2026にアクセス、https://devblogs.microsoft.com/dotnet/performance-improvements-in-net-8/
22. Performance Improvements in .NET 6 - Microsoft Developer Blogs, 5月 10, 2026にアクセス、https://devblogs.microsoft.com/dotnet/performance-improvements-in-net-6/
23. Performance Improvements in .NET 7 - Microsoft Developer Blogs, 5月 10, 2026にアクセス、https://devblogs.microsoft.com/dotnet/performance_improvements_in_net_7/
24. NET 7 is Available Today - Microsoft Developer Blogs, 5月 10, 2026にアクセス、https://devblogs.microsoft.com/dotnet/announcing-dotnet-7/
25. Announcing .NET 6 - The Fastest .NET Yet - Microsoft Developer Blogs, 5月 10, 2026にアクセス、https://devblogs.microsoft.com/dotnet/announcing-net-6/
26. What makes C++ “faster” than C#? : r/csharp - Reddit, 5月 10, 2026にアクセス、https://www.reddit.com/r/csharp/comments/1bkf0c3/what_makes_c_faster_than_c/
27. Native AOT - Static PGO MIBC passed to ILC appears to be ignored · Issue #95236 · dotnet/runtime - GitHub, 5月 10, 2026にアクセス、

https://github.com/dotnet/runtime/issues/95236
28. Cannot reproduce instruction in NativeAOT Support and Precompiled Queries (Experimental) · Issue #35887 · dotnet/efcore - GitHub, 5月 10, 2026にアクセス、https://github.com/dotnet/efcore/issues/35887
29. What's New in EF Core 10 | Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/ef/core/what-is-new/ef-core-10.0/whatsnew
30. Entity Framework Core 10 (EF Core 10) - atal upadhyay - WordPress.com, 5月 10, 2026にアクセス、https://atalupadhyay.wordpress.com/2025/11/21/entity-framework-core-10-ef-core-10/
31. What's New in .NET 10: EF Core, MAUI, AOT & Security - FutureStack Solution, 5月 10, 2026にアクセス、https://www.futurestacksolution.com/whats-new-in-dotnet-10/
32. AI Innovations in Software Development | PDF | Artificial Intelligence - Scribd, 5月 10, 2026にアクセス、https://www.scribd.com/document/884385515/CODE-2025-magazine
33. SqlClient/release-notes/7.0/7.0.0-preview1.md at main · dotnet/SqlClient · GitHub, 5月 10, 2026にアクセス、https://github.com/dotnet/SqlClient/blob/main/release-notes/7.0/7.0.0-preview1.md
34. Plan for Entity Framework Core 7.0 - Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/ef/core/what-is-new/ef-core-7.0/plan
35. Issues · dotnet/SqlClient - GitHub, 5月 10, 2026にアクセス、https://github.com/dotnet/SqlClient/issues
36. Breaking changes in EF Core 11 (EF11) - Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/ef/core/what-is-new/ef-core-11.0/breaking-changes
37. What is the current status of Native AOT? : r/dotnet - Reddit, 5月 10, 2026にアクセス、https://www.reddit.com/r/dotnet/comments/1i40lo3/what_is_the_current_status_of_native_aot/
38. On Web Framework Performance: A .NET 10 Reality Check - DEV Community, 5月 10, 2026にアクセス、https://dev.to/insight105/on-web-framework-performance-a-net-10-reality-check-4ep5
39. .NET 10 and the Next Leap in Native AOT for Cloud-Native Apps - DEV Community, 5月 10, 2026にアクセス、https://dev.to/arthus15/net-10-and-the-next-leap-in-native-aot-for-cloud-native-apps-a0k
40. What's new in ASP.NET Core in .NET 10 - Microsoft Learn, 5月 10, 2026にアクセス、https://learn.microsoft.com/en-us/aspnet/core/release-notes/aspnetcore-10.0?view=aspnetcore-10.0
41. ASP.NET Core Performance Tuning in 2026: Optimizations That Cut Latency, Costs, and CI Time | Syncfusion Blogs, 5月 10, 2026にアクセス、https://www.syncfusion.com/blogs/post/performance-tuning-in-aspnetcore-202

6
42. Edge Cases in .NET Framework 4.x to .NET 10 Migration - GAPVelocity AI, 5月 10, 2026にアクセス、https://www.gapvelocity.ai/blog/edge-cases-in-.net-framework-4.x-to-.net-10-migration